So, what's the algorithm to find the nearest neighbor of a 1-D point, when our dataset is stored in a binary search tree?

1. Run search on the binary search tree.
2. If there is an exact match, the entry found is the nearest neighbor (with distance equal to `0`).
3. Otherwise, check which entry is closest to your target between the last entry visited (the one on which search stopped) and its parent.

Now that we have brilliantly solved the problem in 1-D, the question arises: can we use a similar data structure to solve the problem in 2-D?

8.3 Description and API

The answer is, of course, yes – probably the fact that we asked the question already led you to suspect it. But nonetheless, getting from 1-D to 2-D is a big leap – there is no easy way to imagine a tree that works in two dimensions. Worry not though, we'll get into a detailed explanation in the next section. Once we have taken that leap, it will be easy to move to 3-D and in general to hyper-spaces with an arbitrary number of dimensions.

Even more, we won't be limited to datasets that lie in 2-D or 3-D geometric space: the dimensions can be anything, as long as we can define a distance measure on them, with the caveat that it respects some requirements, namely they need to be Euclidian distance[2]. For example, we can have 2-D entries where the first coordinate is their price and the second coordinate is their rating, and then we can ask for